

DEV FULLSTACK SÊNIOR – BENCORP

Você vai atuar de ponta a ponta, com peso maior no backend. Na prática, modelar e implementar regras de negócio complexas, sustentar as APIs que rodam nossos produtos e também entregar as interfaces que consomem essas APIs, de portais autenticados a landing pages com CRMs de alta performance.

Nossa stack é heterogênea: Node/TypeScript no core transacional, Python onde faz sentido para processamento e integrações e SQL Server. Procuramos alguém confortável em transitar entre esses mundos e, mais importante, capaz de argumentar qual ferramenta usar em cada caso.

RESPONSABILIDADES:

- Projetar e desenvolver APIs e serviços em Node.js/TypeScript e Python
- Traduzir regras de negócio complexas em código sustentável e testável
- Modelar dados e escrever queries performáticas em SQL Server e PostgreSQL
- Desenvolver interfaces em React/Next.js: portais internos e landing pages
- Propor abordagens arquiteturais para evoluir os serviços existentes
- Revisar código do time e apoiar no planejamento e quebra de tarefas
- Identificar e endereçar gargalos de performance e pontos de dívida técnica
- Garantir qualidade das entregas: testes, observabilidade e documentação

REQUISITOS TÉCNICOS OBRIGATÓRIOS:

- Experiência sólida em Node.js com TypeScript (Express, NestJS ou equivalente)
- Experiência com Python
- SQL Server e/ou PostgreSQL, modelagem, índices, otimização de query e migrations versionadas
- React e Next.js, SSR/SSG, gerenciamento de estado, consumo de API
- Docker e containerização de aplicações
- Git com fluxo de trabalho colaborativo (branches, PRs, code review)
- Vivência com metodologias ágeis (Scrum, Kanban)
- Inglês para leitura de documentação técnica

DIFERENCIAL:

- Experiência com C#, Jest (Teste Unitário);
- Arquitetura de microsserviços e comunicação entre serviços (REST, mensageria)
- Testes automatizados (Jest, Pytest) e cultura de testes
- CI/CD e pipelines de deploy
- Observabilidade: logs estruturados, métricas, tracing.

44

CASE TÉCNICO - PRAZO: 05 DIAS CORRIDOS

Em aplicações web, um problema comum de segurança ocorre quando o usuário não possui permissão funcional mas consegue acessar um recurso digitando a URL diretamente ou chamando a rota via API. Em saúde, o impacto disso não é só funcional, é um vazamento de dados sensível.

Você vai construir um **recorte simplificado de uma plataforma de teleatendimento**, uma fila de pronto atendimento digital, onde profissionais de saúde atendem pacientes por videochamada e registram o prontuário. A aplicação deve ser um PWA e nenhuma rota, dado ou sala de vídeo pode ser acessada sem autorização explícita, independentemente do frontend.

PERFIS

- **ENFERMEIRO:** Fila de PAD, iniciar atendimento, triagem, encaminhar para médico.
- **MEDICO:** Fila de PAD, atendimentos encaminhados, prescrição, prontuário completo.
- **ADMIN:** Gestão de usuários e permissões. Não acessa prontuário clínico
- **PACIENTE:** Sem login. Entra na sala por link de acesso temporário

TELAS

1) FILA DE PRONTO ATENDIMENTO

- Colunas: paciente, contato, classificação de risco, status, entrada na fila, tempo de espera.
- Filtros: busca por nome/CPF, status (aguardando · em atendimento · finalizado · cancelado), período (hoje · ontem · última semana · todos).
- Ações: [Iniciar Atendimento] · [Ver Atendimento]

2) SALA DE ATENDIMENTO

- Vídeo + chat de texto + formulário de prontuário lado a lado. Ao finalizar, o profissional decide entre encaminhar para médico ou encerrar.

3) PACIENTES

- Listagem com busca por nome/CPF. Detalhe com histórico de atendimentos, sinais vitais e prontuários anteriores.

REGRAS OBRIGATÓRIAS

- Só um profissional por atendimento. Dois cliques simultâneos em [Iniciar Atendimento]: um vence, o outro recebe 409. Resolver no banco, não com if antes do update.
- Profissional com atendimento EM_ANDAMENTO não pode iniciar outro.
- Estados: AGUARDANDO → EM_ANDAMENTO → FINALIZADO. CANCELADO só a partir de AGUARDANDO. Transição fora do grafo retorna 422.
- O prontuário finalizado é imutável. Correção só via adendo, com autor e timestamp próprios.
- Todo acesso ao prontuário gera log de auditoria (quem, quando, qual paciente, qual endpoint).
- A sala só existe enquanto o atendimento está EM_ANDAMENTO. Token de acesso emitido pelo backend, TTL curto (≤ 15 min, renovável), vinculado ao atendimento e ao participante.
- Paciente entra por link de uso único, com token opaco, expirável e revogável na finalização. Link de outro atendimento retorna 403.
- Finalizar o atendimento invalida todos os tokens de sala imediatamente.

REGRAS GERAIS

- **Utilizar as seguintes tecnologias:**
 - Node.js (NestJS ou Express) com TypeScript · ORM com migrations (TypeORM ou Prisma) · PostgreSQL · JWT · React · Docker.
 - Para o vídeo, use um serviço pronto (LiveKit, Jitsi ou Daily).
- **O código deve seguir** boas práticas como funções coesas, tipagem correta, e separação por responsabilidade.
- **A solução deve rodar em Docker.**
- **O uso de IA é permitido, desde que seja utilizado de forma consciente e bem justificada.** A pessoa deve ter domínio sobre o que foi feito, entender as decisões tomadas e conseguir explicar claramente em quais pontos a IA foi utilizada e como ela contribuiu para o resultado final.
- **É obrigatório subir o código no GitHub**, contendo:
 - README.md com o passo a passo para execução local e via Docker;
 - Estrutura de pastas organizada e clara;
 - Documentação dos trade-offs e das decisões técnicas adotadas;
 - Dockerfile funcional;
 - Explicação das limitações técnicas identificadas e da abordagem escolhida para a implementação.